

Sprint Documentation

Sprint 14	Start Date: 30/03/2026	Final Date: 03/04/2026
Projetc Name:	civitas-backend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Carlos Alberto Ferreira Candido		
João Antonio Gomes Cestonaro		

Sprint Backlog

Task #	Description	Start Date	Final date
#68	Validações dos campos do fornecedor	30/03/2026	03/04/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
#68	Garantir que o cadastro de fornecedores possua validações completas de integridade, consistência e regras de negócio, evitando dados inválidos, duplicidade e problemas fiscais no sistema.	Carlos Alberto Ferreira Candido, João Antonio Gomes Cestonaro	Completo	10 horas	10 horas

Sprint Description

Tarefa: Sprint 14 - Validações dos campos do Fornecedor

Objetivo

Garantir que o cadastro de fornecedores possua validações completas de integridade, consistência e regras de negócio, evitando dados inválidos, duplicidade e problemas fiscais no sistema.

Descrição

Implementar validações no backend para os campos do cadastro de fornecedor, assegurando:

- Dados obrigatórios
- Formatação correta

- Consistência de dados
- Regras fiscais (CNPJ)
- Integridade com entidades relacionadas

Critérios de Aceite

Campos Obrigatórios

-
- Validar que os campos obrigatórios não estão vazios:
 - Todos os campos são obrigatórios

Validação de Nome Fantasia

- Deve possuir no máximo 150 caracteres

Não pode ser vazio

Remover espaços em branco no início/fim (trim)

Validação de Razão Social (Nome)

- Deve possuir no máximo 150 caracteres

Não pode ser vazio

Remover espaços em branco no início/fim (trim)

Validação de CNPJ

- Não pode ser vazio

Deve conter exatamente 14 dígitos após normalização (deve permitir inserção com máscara e remover para salvar no banco)

Remover caracteres especiais (., /, -)

Validar CNPJ e recusar inválido

Deve ser único no sistema (não permitir duplicidade)

Validação de Situação

-
- Aceitar apenas valores válidos:
 - 1 = ATIVO
 - 2 = INATIVO

Utilizar enum para validação

Validação de Endereço

Logradouro

- Máximo de 200 caracteres

Aplicar trim

Número

- Máximo de 20 caracteres

Aplicar trim

Bairro

- Máximo de 100 caracteres

Aplicar trim

Cidade

- Máximo de 100 caracteres

Aplicar trim

Estado (UF)

- Deve conter exatamente 2 caracteres

Deve ser uma UF válida (ex: SP, RJ, MG)

CEP

- Deve conter 8 dígitos após normalização (permitir inserção com máscara e remover para salvar no banco)

Remover caracteres especiais

Validar formato

Validação de Contato

Telefone

- Máximo de 11 caracteres

Validar formato (deve conter 11 dígitos após a normalização e remoção da máscara)

Aplicar trim

Email

- Máximo de 150 caracteres

Validar formato de e-mail

Aplicar trim

Validação de Relacionamentos

- Validar integridade ao vincular com despesas

Garantir consistência com documentos vinculados

Regras de Negócio

Controle de duplicidade

-
- Não permitir cadastro de fornecedor com:
 - Mesmo cnpj

Fornecedor Inativo

- Não permitir uso de fornecedor com situacao = INATIVO em novas despesas

Normalização de Dados

- Aplicar trim em todos os campos string

Armazenar CNPJ apenas com números

Armazenar CEP apenas com números

Padronizar e-mail (lowercase)

Observações e Sugestões

- Utilizar DataAnnotations para validações básicas
- Implementar validações mais complexas (CNPJ, duplicidade) na camada Service
- Validar CNPJ antes de persistir no banco
- Verifique todo o trabalho antes do envio

Sprint Results

Primeiramente foram feitos métodos no service para validar relacionamento.

Civitas.WebAPI > Services > Entities > FornecedorService.cs

```
1  using AutoMapper;
2  using Civitas.WebAPI.Data.Interfaces;
3  using Civitas.WebAPI.Objects.Dtos.Entities;
4  using Civitas.WebAPI.Objects.Enums;
5  using Civitas.WebAPI.Objects.Models;
6  using Civitas.WebAPI.Services.Interfaces;
7  using System.Net.Mail;
8  using System.Text.RegularExpressions;
9
10 namespace Civitas.WebAPI.Services.Entities
11 {
12     /// <summary>
13     /// Serviço responsável pelo gerenciamento de Fornecedores (Credores) do sistema.
14     /// </summary>
15     /// <remarks>
16     /// Finalidade:
17     /// - Centralizar as operações de cadastro e atualização de empresas prestadoras de serviço.
18     ///
19     /// Regras de Negócio:
20     /// - Atua como validador para garantir que apenas fornecedores com dados fiscais consistentes (CNPJ) sejam mar
21     /// - É essencial para a integridade financeira, pois sem um fornecedor válido, não se pode lançar despesas.
22     ///
23     /// Dependências:
24     /// - <see cref="IFornecedorRepository"/>: Persistência de dados.
25     /// - <see cref="IMapper"/>: Transformação de objetos (DTO/Entity).
26     /// </remarks>
27     public class FornecedorService : GenericService<Fornecedor, FornecedorDTO>, IFornecedorService
28     {
29         private readonly IFornecedorRepository _fornecedorRepository;
30         private static readonly HashSet<string> UFsValidas =
31         [
32             "AC", "AL", "AP", "AM", "BA", "CE", "DF", "ES", "GO", "MA", "MT", "MS", "MG",
33             "PA", "PB", "PR", "PE", "PI", "RJ", "RN", "RS", "RO", "RR", "SC", "SP", "SE", "TO"
34         ];
35
36         /// <summary>
37         /// Inicializa o serviço de Fornecedores com as dependências necessárias.
38         /// </summary>
39         /// <param name="fornecedorRepository">Repositório de acesso a dados de fornecedores.</param>
40         /// <param name="mapper">Mapeador de objetos.</param>
41         /// <exception cref="ArgumentNullException">Lançada caso os parâmetros injetados sejam nulos.</exception>
42         public FornecedorService(IFornecedorRepository fornecedorRepository, IMapper mapper)
43             : base(fornecedorRepository, mapper)
44         {
45             _fornecedorRepository = fornecedorRepository;
46         }
47
48         public async Task ValidarCadastroAsync(FornecedorDTO entityDTO, int? id = null)
```

```

public async Task ValidarCadastroAsync(FornecedorDTO entityDTO, int? id = null)
{
    if (entityDTO is null)
    {
        throw new ArgumentException("Dados do fornecedor são obrigatórios.");
    }

    if (id.HasValue && id.Value <= 0)
    {
        throw new ArgumentException("Id do fornecedor inválido.");
    }

    if (id.HasValue)
    {
        var existingEntity = await _fornecedorRepository.GetById(id.Value);
        if (existingEntity is null)
        {
            throw new KeyNotFoundException($"Entidade com id {id.Value} não encontrada.");
        }
    }

    entityDTO.IdFornecedor = id ?? 0;

    NormalizarCampos(entityDTO);
    ValidarCampos(entityDTO);
    await ValidarUnicidadeAsync(entityDTO, id);
}

private async Task ValidarUnicidadeAsync(FornecedorDTO fornecedorDTO, int? ignoreId)
{
    var cnpjDuplicado = await _fornecedorRepository.ExistsByCnpjAsync(fornecedorDTO.Cnpj, ignoreId);
    if (cnpjDuplicado)
    {
        throw new ArgumentException("CNPJ já cadastrado no sistema.");
    }
}

private static void NormalizarCampos(FornecedorDTO fornecedorDTO)
{
    fornecedorDTO.NomeFantasia = Sanitize(fornecedorDTO.NomeFantasia);
    fornecedorDTO.Nome = Sanitize(fornecedorDTO.Nome);
    fornecedorDTO.Logradouro = Sanitize(fornecedorDTO.Logradouro);
    fornecedorDTO.Numero = Sanitize(fornecedorDTO.Numero);
    fornecedorDTO.Bairro = Sanitize(fornecedorDTO.Bairro);
}

```

```

    fornecedorDTO.Bairro = Sanitize(fornecedorDTO.Bairro);
    fornecedorDTO.Email = Sanitize(fornecedorDTO.Email).ToLowerInvariant();
    fornecedorDTO.Cidade = Sanitize(fornecedorDTO.Cidade);
    fornecedorDTO.Estado = Sanitize(fornecedorDTO.Estado).ToUpperInvariant();

    fornecedorDTO.Cnpj = Sanitize(fornecedorDTO.Cnpj, digitsOnly: true);
    fornecedorDTO.Cep = Sanitize(fornecedorDTO.Cep, digitsOnly: true);
    fornecedorDTO.Telefone = Sanitize(fornecedorDTO.Telefone, digitsOnly: true);
}

private static void ValidarCampos(FornecedorDTO fornecedorDTO)
{
    ValidarObrigatorio(fornecedorDTO.NomeFantasia, nameof(fornecedorDTO.NomeFantasia));
    ValidarObrigatorio(fornecedorDTO.Nome, nameof(fornecedorDTO.Nome));
    ValidarObrigatorio(fornecedorDTO.Cnpj, nameof(fornecedorDTO.Cnpj));
    ValidarObrigatorio(fornecedorDTO.Logradouro, nameof(fornecedorDTO.Logradouro));
    ValidarObrigatorio(fornecedorDTO.Numero, nameof(fornecedorDTO.Numero));
    ValidarObrigatorio(fornecedorDTO.Bairro, nameof(fornecedorDTO.Bairro));
    ValidarObrigatorio(fornecedorDTO.Cidade, nameof(fornecedorDTO.Cidade));
    ValidarObrigatorio(fornecedorDTO.Estado, nameof(fornecedorDTO.Estado));
    ValidarObrigatorio(fornecedorDTO.Cep, nameof(fornecedorDTO.Cep));
    ValidarObrigatorio(fornecedorDTO.Telefone, nameof(fornecedorDTO.Telefone));
    ValidarObrigatorio(fornecedorDTO.Email, nameof(fornecedorDTO.Email));

    ValidarTamanhoMaximo(fornecedorDTO.NomeFantasia, 150, "NomeFantasia");
    ValidarTamanhoMaximo(fornecedorDTO.Nome, 150, "Nome");
    ValidarTamanhoMaximo(fornecedorDTO.Logradouro, 200, "Logradouro");
    ValidarTamanhoMaximo(fornecedorDTO.Numero, 20, "Numero");
    ValidarTamanhoMaximo(fornecedorDTO.Bairro, 100, "Bairro");
    ValidarTamanhoMaximo(fornecedorDTO.Cidade, 100, "Cidade");
    ValidarTamanhoMaximo(fornecedorDTO.Email, 150, "Email");

    if (fornecedorDTO.IdFornecedor < 0)
    {
        throw new ArgumentException("IdFornecedor não pode ser negativo.");
    }

    var valorSituacao = (int)fornecedorDTO.Situacao;
    if (valorSituacao is not ((int)Situacao.ATIVO) and not ((int)Situacao.INATIVO))
    {
        throw new ArgumentException("Situação inválida. Valores permitidos: 1 (Ativo) ou 2 (Inativo).");
    }

    if (fornecedorDTO.Cnpj.Length != 14 || !IsCnpj(fornecedorDTO.Cnpj))
    {
        throw new ArgumentException("CNPJ inválido.");
    }
}

```

```

    }

    if (fornecedorDTO.Cep.Length != 8)
    {
        throw new ArgumentException("CEP deve conter 8 dígitos.");
    }

    if (fornecedorDTO.Telefone.Length != 11)
    {
        throw new ArgumentException("Telefone deve conter 11 dígitos com DDD.");
    }

    if (fornecedorDTO.Estado.Length != 2 || !UFsValidas.Contains(fornecedorDTO.Estado))
    {
        throw new ArgumentException("Estado inválido. Informe uma UF válida.");
    }

    if (!IsEmailValido(fornecedorDTO.Email))
    {
        throw new ArgumentException("E-mail inválido.");
    }
}

private static void ValidarObrigatorio(string valor, string campo)
{
    if (string.IsNullOrEmpty(valor))
    {
        throw new ArgumentException($"Campo obrigatório não preenchido: {campo}.");
    }
}

private static void ValidarTamanhoMaximo(string valor, int maximo, string campo)
{
    if (valor.Length > maximo)
    {
        throw new ArgumentException($"Campo {campo} deve conter no máximo {maximo} caracteres.");
    }
}

private static bool IsEmailValido(string email)
{
    try
    {
        var mailAddress = new MailAddress(email);
    }
}

```



```

177     {
178         try
179         {
180             var mailAddress = new MailAddress(email);
181             return mailAddress.Address == email;
182         }
183         catch
184         {
185             return false;
186         }
187     }
188
189     private static string Sanitize(string? valor, bool digitsOnly = false)
190     {
191         if (string.IsNullOrEmpty(valor))
192         {
193             return string.Empty;
194         }
195
196         var sanitized = valor.Trim();
197         return digitsOnly
198             ? Regex.Replace(sanitized, "[^0-9]", string.Empty)
199             : sanitized;
200     }
201
202     private static bool IsCnpj(string cnpj)
203     {
204         if (cnpj.Length != 14)
205         {
206             return false;
207         }
208
209         if (cnpj.All(caractere => caractere == cnpj[0]))
210         {
211             return false;
212         }
213
214         int[] pesoPrimeiroDigito = [5, 4, 3, 2, 9, 8, 7, 6, 5, 4, 3, 2];
215         int[] pesoSegundoDigito = [6, 5, 4, 3, 2, 9, 8, 7, 6, 5, 4, 3, 2];
216
217         var somaPrimeiroDigito = 0;
218         for (var i = 0; i < 12; i++)
219         {
220             somaPrimeiroDigito += (cnpj[i] - '0') * pesoPrimeiroDigito[i];
221         }
222

```

```

217         var somaPrimeiroDigito = 0;
218         for (var i = 0; i < 12; i++)
219         {
220             somaPrimeiroDigito += (cnpj[i] - '0') * pesoPrimeiroDigito[i];
221         }
222
223         var restoPrimeiroDigito = somaPrimeiroDigito % 11;
224         var digito13 = restoPrimeiroDigito < 2 ? 0 : 11 - restoPrimeiroDigito;
225
226         var somaSegundoDigito = 0;
227         for (var i = 0; i < 13; i++)
228         {
229             var numero = i == 12 ? digito13 : cnpj[i] - '0';
230             somaSegundoDigito += numero * pesoSegundoDigito[i];
231         }
232
233         var restoSegundoDigito = somaSegundoDigito % 11;
234         var digito14 = restoSegundoDigito < 2 ? 0 : 11 - restoSegundoDigito;
235
236         return cnpj[12] - '0' == digito13 && cnpj[13] - '0' == digito14;
237     }
238 }
239 }
240

```

Depois foi declarado o método na interface de fornecedor

Civitas.WebAPI > Services > Interfaces > IFornecedorService.cs

```
1 using Civitas.WebAPI.Objects.Dtos.Entities;
2 using Civitas.WebAPI.Objects.Models;
3
4 namespace Civitas.WebAPI.Services.Interfaces
5 {
6     /// <summary>
7     /// Interface que define o contrato para o serviço de gerenciamento de Fornecedores (Credores).
8     /// </summary>
9     /// <remarks>
10    /// Herda as operações de <see cref="IGenericService{Fornecedor, FornecedorDTO}"/>.
11    /// Utilizada via injeção de dependência para aplicar regras de negócio sobre as empresas prestadoras de serviço,
12    /// garantindo que apenas fornecedores válidos sejam vinculados às despesas.
13    /// </remarks>
14    public interface IFornecedorService : IGenericService<Fornecedor, FornecedorDTO>
15    {
16        Task ValidarCadastroAsync(FornecedorDTO entityDTO, int? id = null);
17    }
18 }
19
```

Assinaturas